

Data Mining with Apriori and Brute Force Algorithms – A Comparison

Alex C. Berger

New Jersey Institute of Technology

Ying Wu College of Computing

October 12th, 2024

Table of Contents

Introduction.....	3
Methodology.....	4-6
Data Description.....	7-14
Apriori Algorithm Implementation.....	15-22
Brute Force Algorithm Implementation.....	23-31
Results Comparison.....	32-33
Conclusion.....	34-35
References.....	36
Source Code.....	37-49

Introduction

Association rule mining is a fundamental data mining technique used to uncover relationships between items in large datasets. It is widely used in market basket analysis, where the goal is to find associations between products purchased together by customers. These associations provide valuable insights into consumer behavior and help businesses make data-driven decisions to improve marketing strategies, cross-selling, and inventory management.

Two common methods that are used to discover frequent itemsets and generate association rules are the **Apriori algorithm** and the **brute force algorithm**. Both aim to identify relationships between items in a transaction database, but they differ in terms of efficiency and scalability.

The Apriori algorithm is an efficient, bottom-up approach that leverages the "Apriori property," which states that any subset of a frequent itemset must also be frequent. By using this property, the algorithm reduces the number of itemsets that need to be checked, significantly improving performance for large datasets. It operates by first finding the frequent itemsets and then using these itemsets to generate association rules.

In contrast, the brute force method takes a more exhaustive approach. The algorithm generates all possible itemsets and checks the frequency of each one, making it computationally expensive, especially for larger datasets. Although the brute force algorithm guarantees finding all frequent itemsets, it becomes impractical for large datasets due to the mass combinations of itemsets.

This project aims to implement both the Apriori algorithm and the brute force method on the same datasets, compare their performance in terms of computation time, and evaluate their ability to generate the same association rules. Five transaction databases, each containing 20 transactions, were created and used for this analysis.

Methodology

Data Preparation

For this project, five distinct transaction databases were created, each containing 20 transactions. These databases simulate typical market basket data, where each transaction represents a customer's purchase containing multiple items. A total of 30 unique products were used across these transactions. The products range from common household items like bread, milk, and eggs to non-food items such as diapers, shampoo, and toilet paper.

The transaction data for each database was stored in a relational database management system using Azure Data Studio. Each database includes two tables:

1. **Products** – a list of 30 unique product items.
2. **Transactions** – a table where each row corresponds to a single product within a specific transaction.

Algorithms Implemented

The goal of the analysis was to generate association rules from the transaction data using two different methods: the **Apriori algorithm** and the **brute force algorithm**.

1. Apriori Algorithm

The Apriori algorithm was applied to each of the five databases to generate frequent itemsets and association rules. The algorithm works in a level-wise manner by first generating 1-itemsets, then k-itemsets, while pruning itemsets that do not meet the minimum support threshold. The steps involved in applying Apriori were:

- **Preprocessing the Data:** Each database was queried using SQL to retrieve transaction data. The data was then transformed into a basket format, where each row represents a transaction, and each column represents a product. The presence of a product in a transaction was indicated by a binary value (1 for present, 0 for absent) then converted to Boolean type for library purposes.
- **Setting Parameters:** Support and confidence values were given as key parameters. Different values for support and confidence were tested across the five databases to identify the best rules.
- **Generating Frequent Itemsets:** Using the mlxtend Python library, frequent itemsets were generated based on the minimum support threshold.
- **Generating Association Rules:** Association rules were then extracted from the frequent itemsets by specifying a minimum confidence level. These rules were evaluated based on two metrics; **support** and **confidence**.

2. Brute Force Method

The brute force method was implemented to serve as a comparison with the Apriori algorithm. The brute force method checks all possible itemsets to identify frequent ones. The steps were as follows:

- **Preprocessing the Data:** Similar to the Apriori method, the transaction data was formatted as a basket of binary values.
- **Generating Itemsets:** All possible itemsets (1-itemsets, 2-itemsets, 3-itemsets, etc.) were generated using the combinations function from Python's itertools library.
- **Support Calculation:** For each itemset, the number of transactions in which the itemset appeared was counted, and the support was calculated as the ratio of this count to the total number of transactions.
- **Identifying Frequent Itemsets:** Itemsets meeting the minimum support threshold were identified as frequent itemsets.
- **Rule Generation:** Similar to the Apriori method, association rules were generated for the frequent itemsets.

Evaluation and Comparison

Both methods were applied to each of the five transaction databases. The results from the brute force method were compared to those generated by the Apriori algorithm to ensure consistency in the association rules produced. The main metrics of interest for this comparison were:

- **Frequent Itemsets:** The sets of items that appeared together frequently in the transactions.
- **Association Rules:** The rules derived from the frequent itemsets, showing relationships between products.
- **Performance (Computation Time):** A critical factor in evaluating the efficiency of the two methods was their computation time. The brute force method, given its exhaustive approach, was expected to take significantly longer than the Apriori algorithm, especially for larger datasets.

Tools and Libraries

- **Microsoft SQL Server:** For storing and querying the transaction databases via SQL.
- **Azure Data Studio:** IDE for database querying and server management.
- **Python:** Used for data processing, algorithm implementation, and rule generation.
- **mlxtend Library:** This Python library was used to implement the Apriori algorithm and generate association rules.
- **SQLAlchemy Library:** A Python SQL toolkit used for connecting to and querying the SQL databases.
- **Time Library:** A Python library used for calculating run-times.

File Edit View Help

localhost:transactionalDB1 X

Home > localhost > TransactionalDB1

Recovery Model : Full
Last Log Backup : Never
Owner : Alex-Desktoplaberg

Last Database Backup : 10/12/2024 6:50:00 AM
Compatibility Level : 100

Search

Search by name of type (t, v, f, or sp)

Name	Schema	Type	Actions
Products	dbo	Table	...
Transactions	dbo	Table	...

AZURE

Choose SQL Language localhost : TransactionalDB1

Data Description

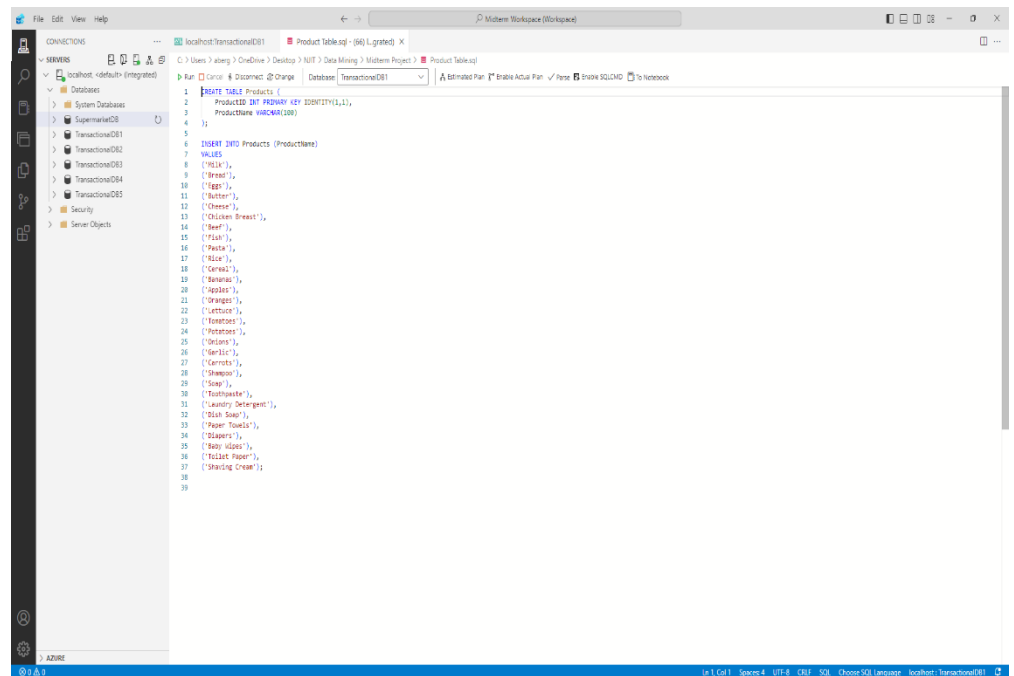
The data used in this project simulates market basket transactions, representing typical purchases made by customers at a supermarket. The dataset includes **five distinct transaction databases**, each containing 20 transactions. The products within these transactions are drawn from a set of 30 unique items commonly found in supermarkets.

Product List

The 30 products used in the transactions can be grouped into various categories, as outlined below:

- **Groceries:**

- Milk
- Bread
- Eggs
- Butter
- Cheese
- Chicken Breast
- Beef
- Fish
- Pasta
- Rice
- Cereal
- Bananas
- Apples
- Oranges
- Lettuce
- Tomatoes
- Potatoes
- Onions
- Garlic
- Carrots



- **Household Items:**
 - Shampoo
 - Soap
 - Toothpaste
 - Laundry Detergent
 - Dish Soap
 - Paper Towels
 - Diapers
 - Baby Wipes
 - Toilet Paper
 - Shaving Cream

Each product has a **Product ID**, a unique identifier used in the database, and a **Product Name** for easy referencing. These products are part of the **Products** table within each database.

Transaction Data

Each of the five databases contains 20 transactions, and each transaction represents a customer purchase consisting of three of the products listed above. The data for the transactions is stored in the **Transactions** table in each database. Each transaction is represented by the following attributes:

- **Transaction ID:** A unique identifier for each transaction.
- **Product ID:** Refers to the unique identifier of the product in the Products table.

The relationship between the **Transactions** and **Products** tables is a **one-to-many** relationship, as each product can appear in multiple transactions and products are unique within their table.

Transaction Distribution

The transactions were designed to reflect typical shopping patterns and relationships between items. For instance:

- Some transactions feature a combination of related products, such as **bread** and **milk**, or **eggs** and **butter**, which commonly appear together in customer purchases.
- Household products like **soap**, **toilet paper**, and **shampoo** were grouped in certain transactions to reflect customer habits of purchasing these items together.

This underlying structure provides a foundation for generating frequent itemsets and association rules, as patterns of co-occurrence between products are expected to emerge during analysis.

Summary of Datasets

Each of the five transaction databases contains the following:

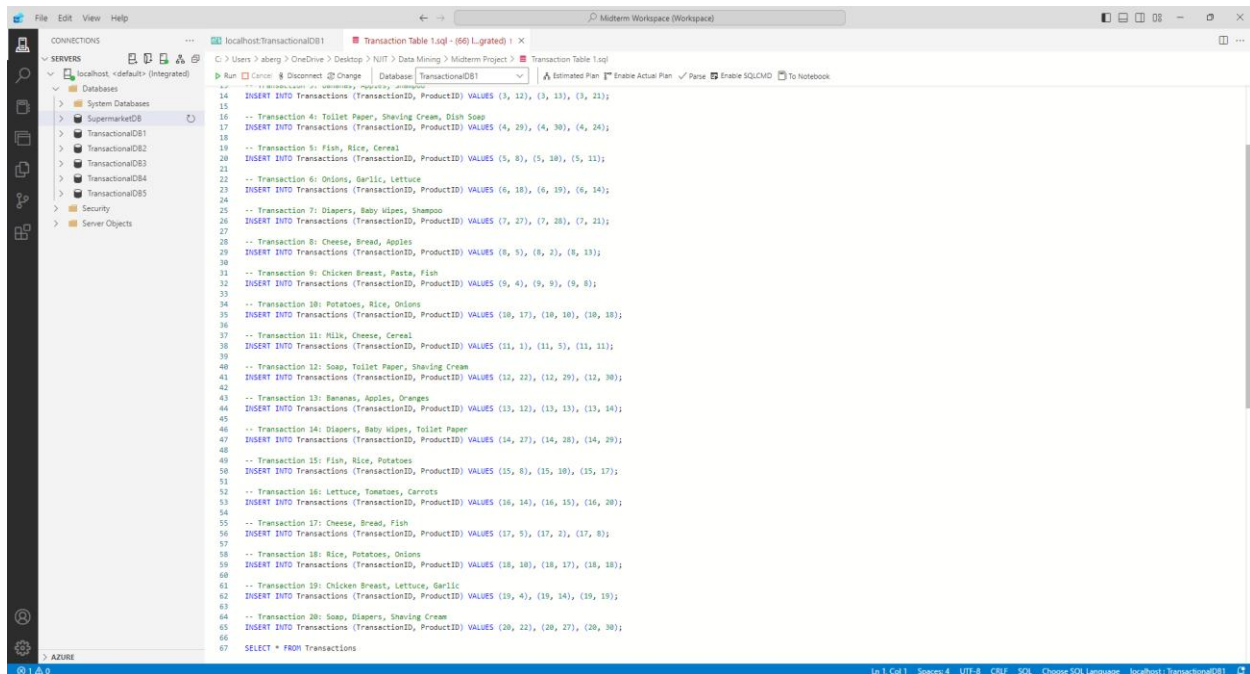
- **20 Transactions:** A list of customer purchases containing combinations of products.
 - **60 Total Records:** Each transaction includes three items
- **30 Products:** A unique list of items that are bought across all transactions.

The transaction data is stored in a relational format, where the **Transactions** and **Products** tables are joined to perform analysis. This structure ensures that both the brute force method and the Apriori algorithm can efficiently process and analyze the data.

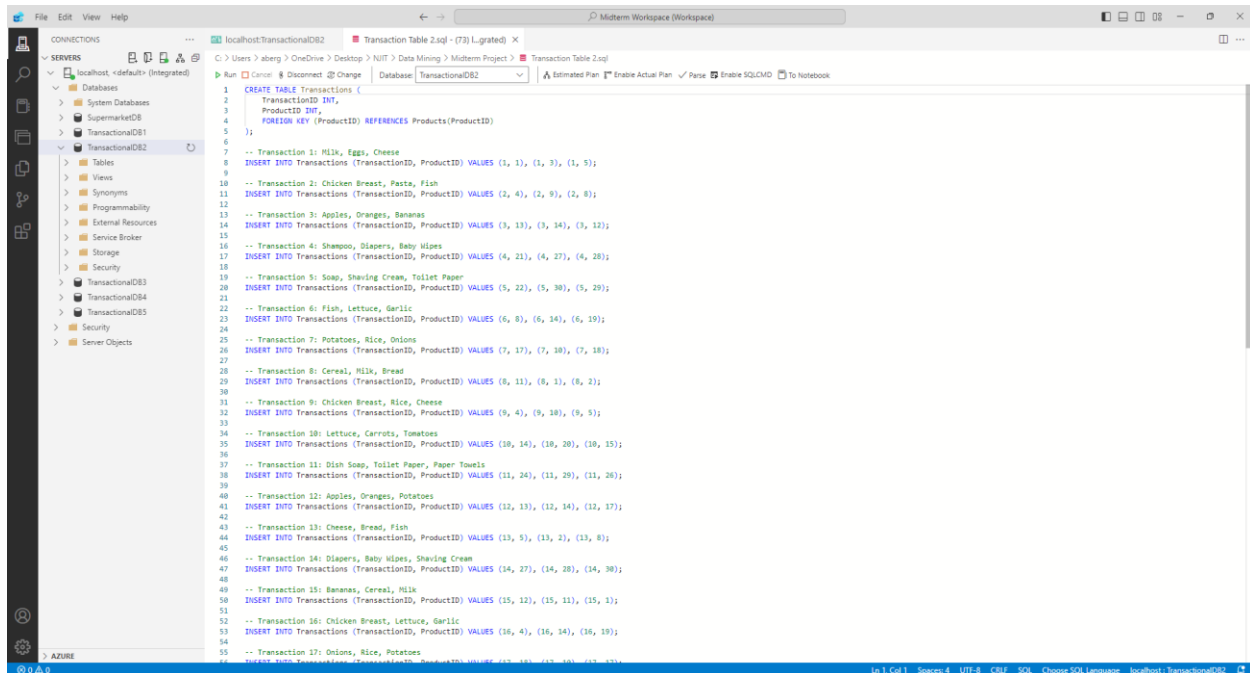
Database One: TransactionalDB1

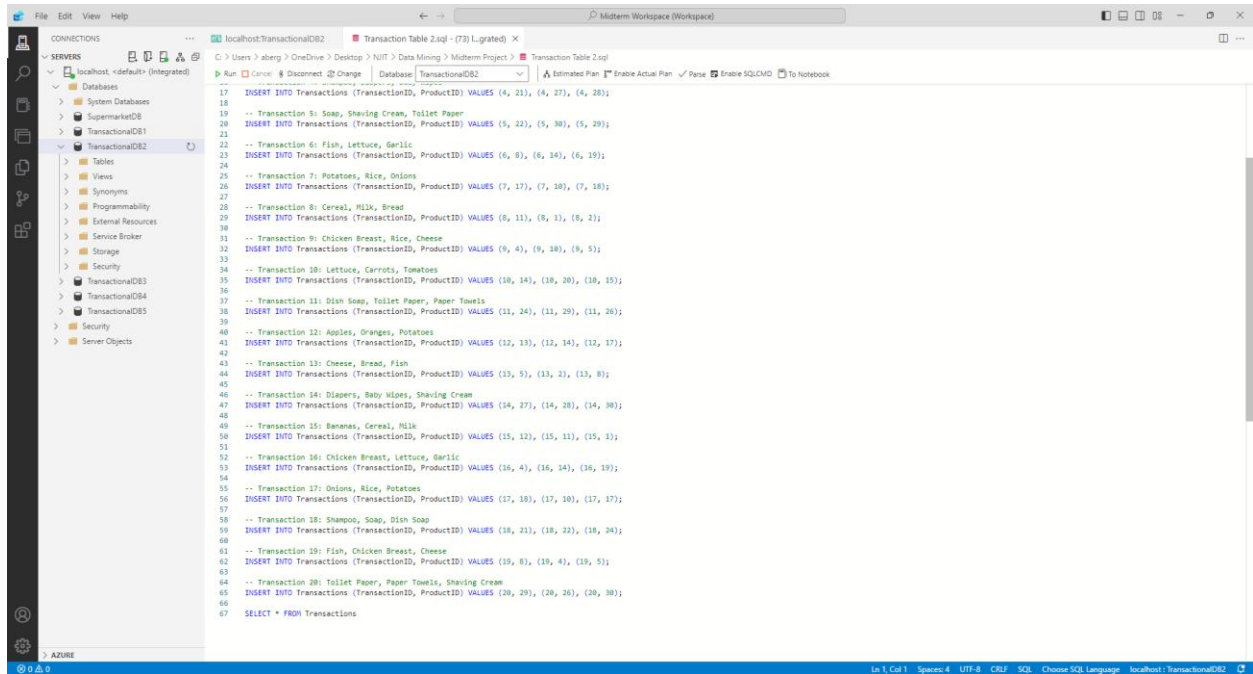
```

1 CREATE TABLE Transactions (
2   TransactionID INT,
3   ProductID INT,
4   FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
5 );
6
7 -- Transaction 1: Milk, Bread, Eggs
8 INSERT INTO Transactions (TransactionID, ProductID) VALUES (1, 1), (1, 2), (1, 3);
9
10 -- Transaction 2: Chicken Breast, Soap, Cheese
11 INSERT INTO Transactions (TransactionID, ProductID) VALUES (2, 4), (2, 22), (2, 5);
12
13 -- Transaction 3: Bananas, Apples, Shampoo
14 INSERT INTO Transactions (TransactionID, ProductID) VALUES (3, 12), (3, 13), (3, 21);
15
16 -- Transaction 4: Toilet Paper, Shaving Cream, Dish Soap
17 INSERT INTO Transactions (TransactionID, ProductID) VALUES (4, 29), (4, 30), (4, 34);
18
19 -- Transaction 5: Fish, Rice, Cereal
20 INSERT INTO Transactions (TransactionID, ProductID) VALUES (5, 8), (5, 10), (5, 11);
21
22 -- Transaction 6: Onions, Garlic, Lettuce
23 INSERT INTO Transactions (TransactionID, ProductID) VALUES (6, 18), (6, 19), (6, 14);
24
25 -- Transaction 7: Diapers, Baby Wipes, Shampoo
26 INSERT INTO Transactions (TransactionID, ProductID) VALUES (7, 27), (7, 28), (7, 21);
27
28 -- Transaction 8: Cheese, Bread, Apples
29 INSERT INTO Transactions (TransactionID, ProductID) VALUES (8, 5), (8, 2), (8, 13);
30
31 -- Transaction 9: Chicken Breast, Pasta, Fish
32 INSERT INTO Transactions (TransactionID, ProductID) VALUES (9, 4), (9, 9), (9, 8);
33
34 -- Transaction 10: Potatoes, Rice, Onions
35 INSERT INTO Transactions (TransactionID, ProductID) VALUES (10, 17), (10, 10), (10, 18);
36
37 -- Transaction 11: Milk, Cheese, Cereal
38 INSERT INTO Transactions (TransactionID, ProductID) VALUES (11, 1), (11, 5), (11, 11);
39
40 -- Transaction 12: Soap, Toilet Paper, Shaving Cream
41 INSERT INTO Transactions (TransactionID, ProductID) VALUES (12, 22), (12, 20), (12, 30);
42
43 -- Transaction 13: Bananas, Apples, Oranges
44 INSERT INTO Transactions (TransactionID, ProductID) VALUES (13, 12), (13, 13), (13, 14);
45
46 -- Transaction 14: Diapers, Baby Wipes, Toilet Paper
47 INSERT INTO Transactions (TransactionID, ProductID) VALUES (14, 27), (14, 28), (14, 29);
48
49 -- Transaction 15: Fish, Rice, Potatoes
50 INSERT INTO Transactions (TransactionID, ProductID) VALUES (15, 8), (15, 10), (15, 17);
51
52 -- Transaction 16: Lettuce, Tomatoes, Carrots
53 INSERT INTO Transactions (TransactionID, ProductID) VALUES (16, 14), (16, 15), (16, 20);
54
55 -- Transaction 17: Cheese, Bread, Fish
56 INSERT INTO Transactions (TransactionID, ProductID) VALUES (17, 5), (17, 2), (17, 8);
  
```

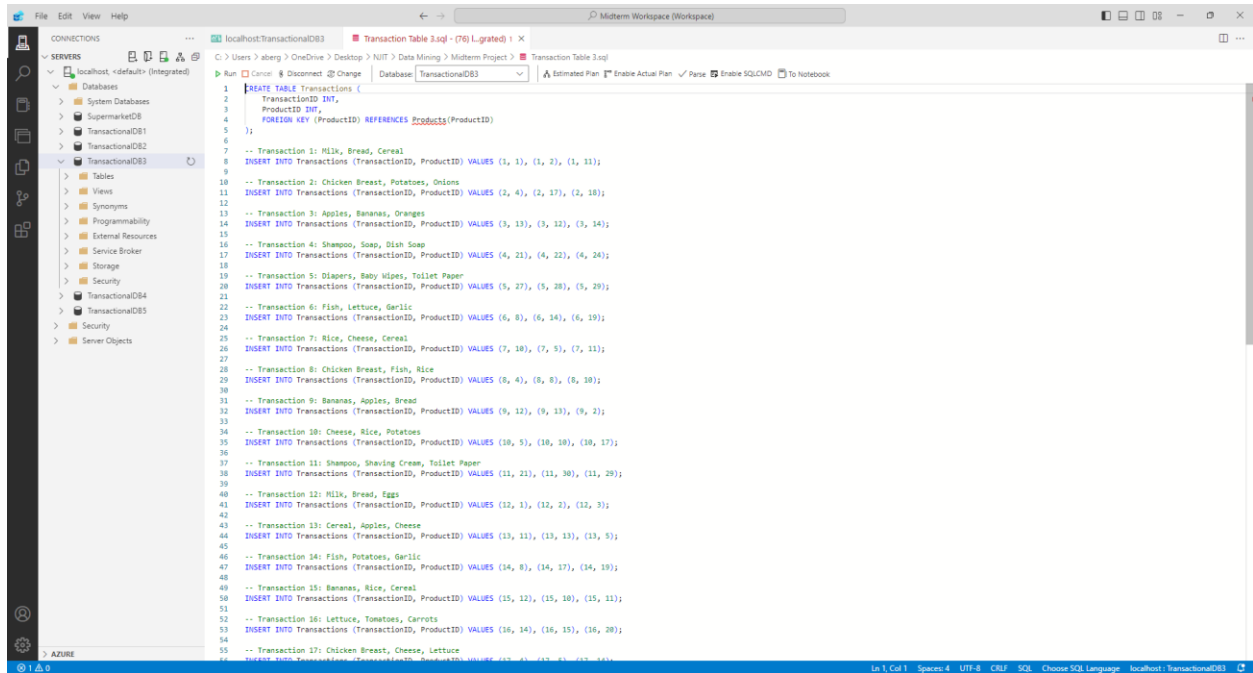


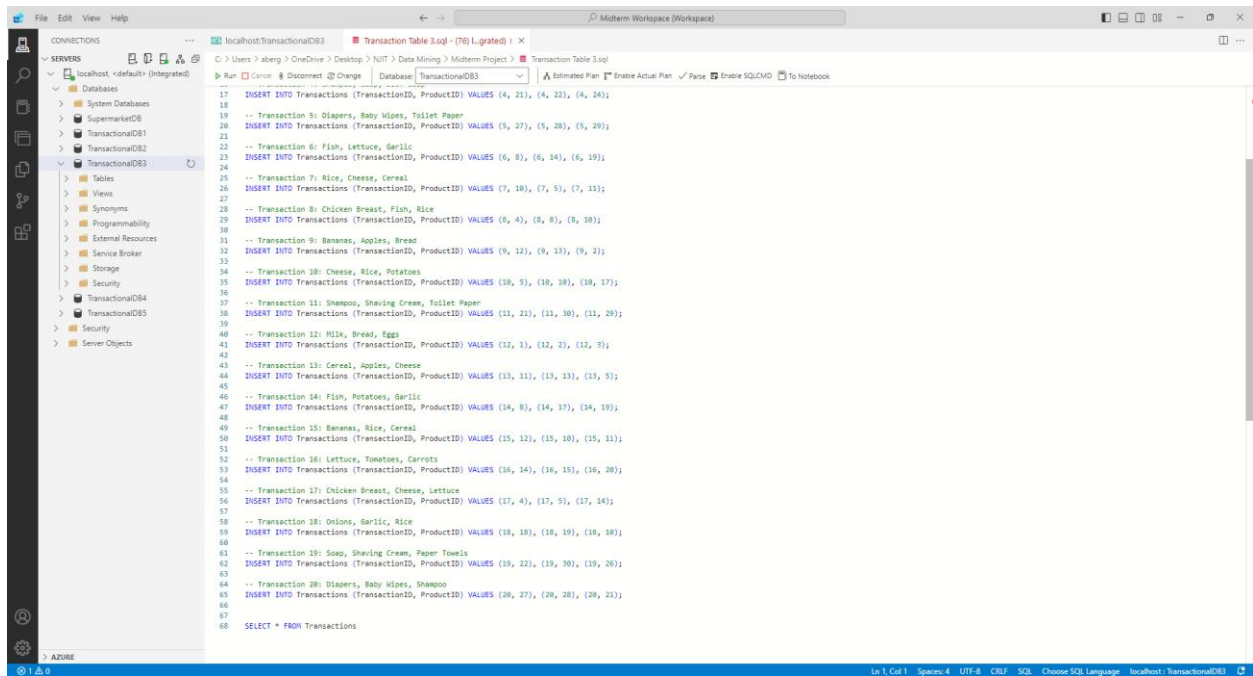
Database Two: TransactionalDB2



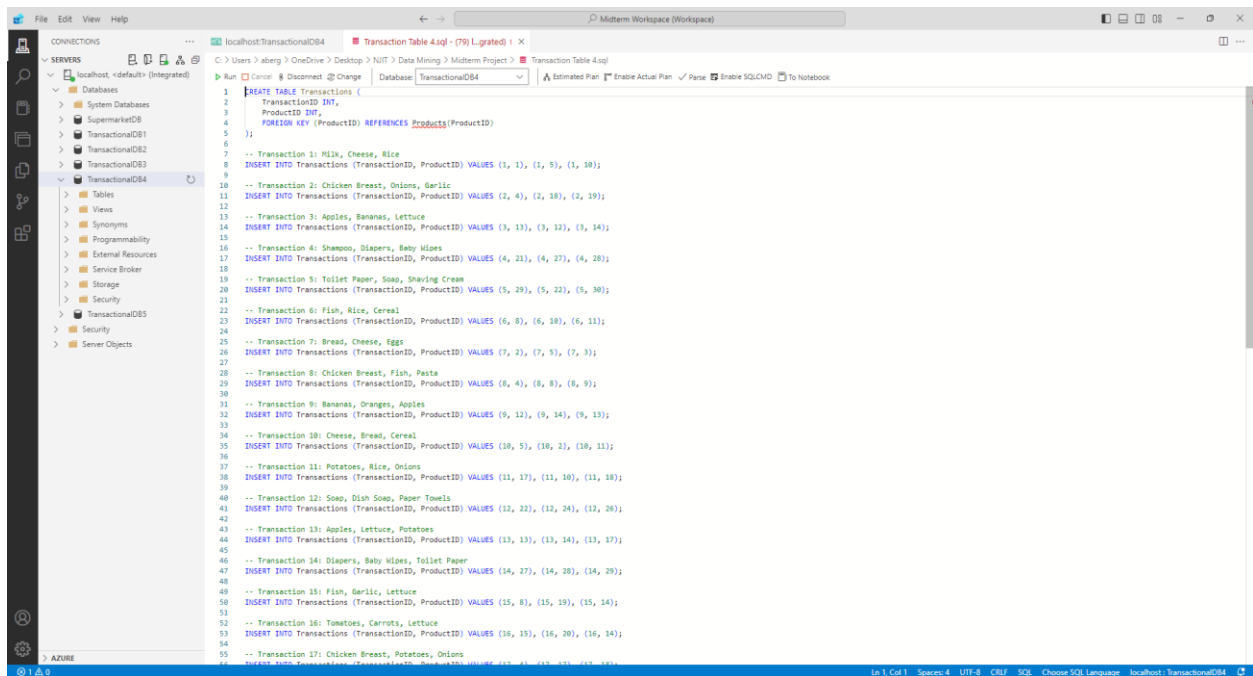


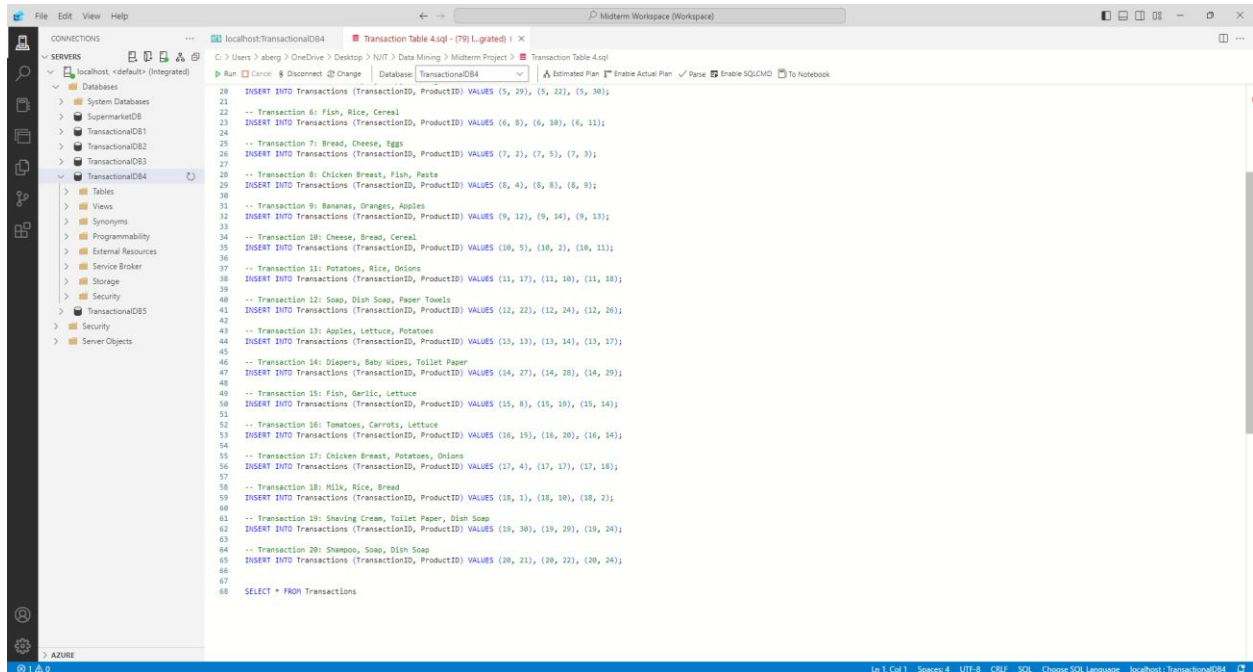
Database Three: TransactionalDB3



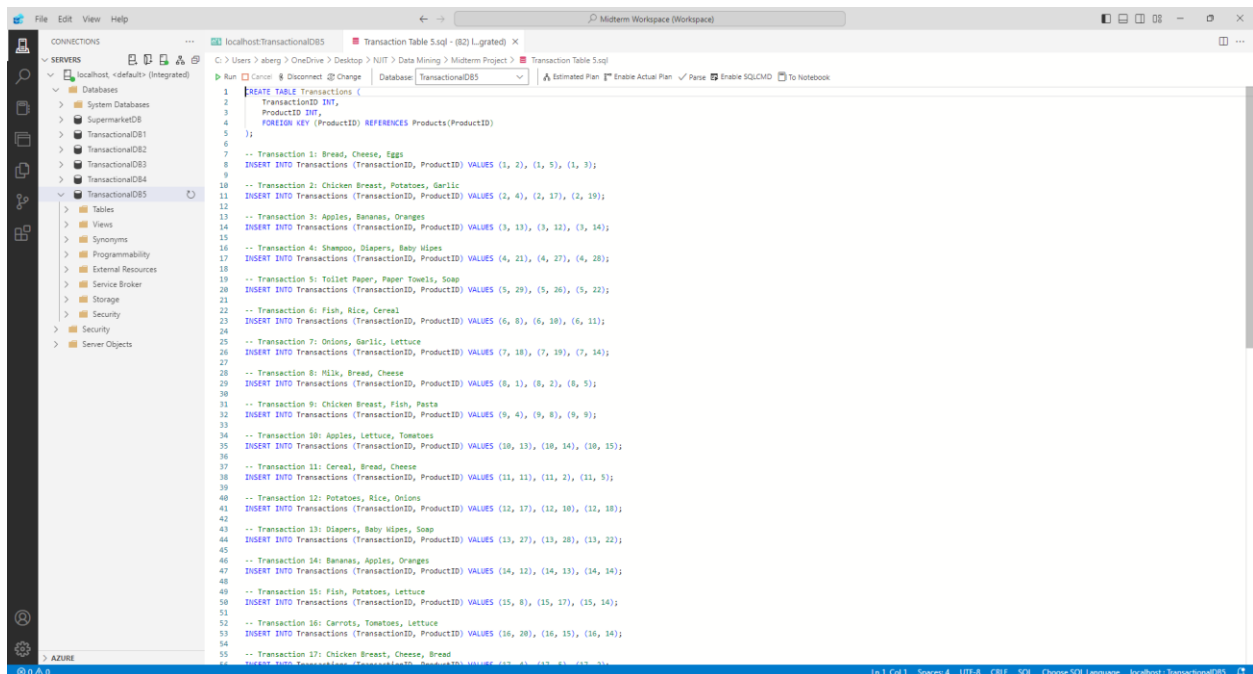


Database Four: TransactionalDB4





Database Five: TransactionalDB5



The screenshot displays the Midterm Workspace (Workspace) application interface. The left sidebar shows a tree view of the database structure, including System Databases, TransactionalDB1, TransactionalDB2, TransactionalDB3, TransactionalDB4, and TransactionalDB5. The main editor area shows a SQL script titled "Transaction Table Sql - (B2) L-graded" with the following content:

```
21 INSERT INTO Transactions (TransactionID, ProductID) VALUES (5, 29), (5, 28), (5, 22);
22
23 -- Transaction 6: Fish, Rice, Cereal
24 INSERT INTO Transactions (TransactionID, ProductID) VALUES (6, 8), (6, 18), (6, 11);
25
26 -- Transaction 7: Onions, Garlic, Lettuce
27 INSERT INTO Transactions (TransactionID, ProductID) VALUES (7, 18), (7, 19), (7, 14);
28
29 -- Transaction 8: Milk, Bread, Cheese
30 INSERT INTO Transactions (TransactionID, ProductID) VALUES (8, 1), (8, 2), (8, 5);
31
32 -- Transaction 9: Chicken Breast, Fish, Pasta
33 INSERT INTO Transactions (TransactionID, ProductID) VALUES (9, 4), (9, 8), (9, 9);
34
35 -- Transaction 10: Apples, Lettuce, Tomatoes
36 INSERT INTO Transactions (TransactionID, ProductID) VALUES (10, 13), (10, 14), (10, 15);
37
38 -- Transaction 11: Cereal, Bread, Cheese
39 INSERT INTO Transactions (TransactionID, ProductID) VALUES (11, 11), (11, 2), (11, 9);
40
41 -- Transaction 12: Potatoes, Rice, Onions
42 INSERT INTO Transactions (TransactionID, ProductID) VALUES (12, 17), (12, 18), (12, 18);
43
44 -- Transaction 13: Diapers, Baby Wipes, Soap
45 INSERT INTO Transactions (TransactionID, ProductID) VALUES (13, 27), (13, 28), (13, 22);
46
47 -- Transaction 14: Bananas, Apples, Oranges
48 INSERT INTO Transactions (TransactionID, ProductID) VALUES (14, 12), (14, 13), (14, 14);
49
50 -- Transaction 15: Fish, Potatoes, Lettuce
51 INSERT INTO Transactions (TransactionID, ProductID) VALUES (15, 8), (15, 17), (15, 14);
52
53 -- Transaction 16: Carrots, Tomatoes, Lettuce
54 INSERT INTO Transactions (TransactionID, ProductID) VALUES (16, 28), (16, 15), (16, 14);
55
56 -- Transaction 17: Chicken Breast, Cheese, Bread
57 INSERT INTO Transactions (TransactionID, ProductID) VALUES (17, 4), (17, 5), (17, 2);
58
59 -- Transaction 18: Garlic, Onions, Rice
60 INSERT INTO Transactions (TransactionID, ProductID) VALUES (18, 19), (18, 18), (18, 18);
61
62 -- Transaction 19: Shampoo, Shaving Cream, Toilet Paper
63 INSERT INTO Transactions (TransactionID, ProductID) VALUES (19, 21), (19, 30), (19, 29);
64
65 -- Transaction 20: Soap, Paper Towels, Dish Soap
66 INSERT INTO Transactions (TransactionID, ProductID) VALUES (20, 22), (20, 26), (20, 24);
67
68 SELECT * FROM Transactions
```

The status bar at the bottom indicates the current file is "Localhost - TransactionalDB5" and the editor is in "SQL" mode.

Apriori Algorithm Implementation

The Apriori algorithm is a widely used method for mining frequent itemsets and discovering association rules in transactional datasets. It works by identifying frequent itemsets based on a given minimum support threshold and then deriving association rules from these itemsets by applying a confidence threshold.

In this project, the Apriori algorithm was applied to five transactional databases. The process involved fetching the transactions from each database, preprocessing them into a suitable format, and running the Apriori algorithm to find frequent itemsets and association rules. The Apriori implementation involved the following steps:

Fetching Transactions

I used SQL queries to fetch transaction data from each of the five databases. The transaction data contained two main columns: **TransactionID** and **ProductName**. This step ensured that all the necessary transaction data was loaded into a Pandas DataFrame for further processing.

```

5
6 # Function to fetch transactions from a specific database
7 def fetch_transactions(db_name):
8     try:
9         connection_string = f'mssql+pyodbc://localhost/{db_name}?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes'
10        engine = sa.create_engine(connection_string)
11        df = pd.read_sql("SELECT t.TransactionID, p.ProductName FROM Transactions t JOIN Products p ON t.ProductID = p.ProductID", engine)
12        return df
13    except Exception as e:
14        print(f"Error reading data from {db_name}:", e)
15        return None
16

```

Preprocessing the Data

The transaction data was converted into a binary format, where each row represented a transaction, and each column represented a product. The value 1 indicated that a product was purchased in the transaction, while 0 indicated it was not. This binary matrix was then changed into a Boolean matrix which is essential for applying the Apriori algorithm from the imported library.

```

16
17 # Preprocessing function for Apriori
18 def preprocess_transactions(df):
19     basket = df.groupby(['TransactionID', 'ProductName'])['ProductName'].count().unstack().reset_index().fillna(0).set_index('TransactionID')
20     basket = basket.map(lambda x: 1 if x > 0 else 0).astype(bool)
21     return basket
22

```

Applying the Apriori Algorithm

Once the transaction data was preprocessed, the Apriori algorithm was applied to find frequent itemsets that met a predefined minimum support threshold. The mlxtend library was used to implement the Apriori algorithm, and association rules were generated using a minimum confidence threshold.


```

23 # Apply Apriori algorithm and time the execution
24 def apply_apriori(basket, min_support, min_confidence):
25     start_time = time.time() # Start timing
26     frequent_itemsets = apriori(basket, min_support=min_support, use_colnames=True)
27
28     if frequent_itemsets.empty():
29         print(f"No frequent itemsets for support={min_support} and confidence={min_confidence}")
30         return frequent_itemsets, pd.DataFrame(), time.time() - start_time # Return empty DataFrame and time
31
32     rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=min_confidence)
33
34     end_time = time.time() # End timing
35     execution_time = end_time - start_time
36
37     return frequent_itemsets, rules, execution_time
38

```

Initially, the implementation of the Apriori algorithm included a process to find the best support and confidence values through an iterative search. The goal was to identify the optimal threshold values for each database that would result in the highest lift for association rules. While this approach was beneficial for fine-tuning the algorithm, it was computationally expensive and did not provide a significant performance improvement for this specific dataset.

To simplify the implementation and focus on the core functionality of the Apriori algorithm, I decided to drop the best coefficient search and instead use predefined support and confidence values across all databases. This change allowed me to streamline the algorithm while maintaining a balance between frequent itemset generation and rule discovery.

In the final version, I chose a **support value of 0.1** and a **confidence value of 0.6**. These values were selected based on preliminary testing, ensuring that they could generate frequent itemsets and association rules for each transactional database without the need for exhaustive coefficient searching. By fixing these threshold values, the algorithm could directly apply the Apriori process to each transactional database, calculate the frequent itemsets, and generate association rules, all while measuring the execution time to compare performance.

```

39 # Function to evaluate predefined support and confidence values for a database
40 def evaluate_apriori(db_name, min_support=0.1, min_confidence=0.6):
41     print(f"\nProcessing {db_name} with Apriori algorithm...")
42     df = fetch_transactions(db_name)
43
44     if df is None:
45         return None
46
47     basket = preprocess_transactions(df)
48
49     # Apply the Apriori algorithm and measure time
50     freq_itemsets, rules, apriori_time = apply_apriori(basket, min_support, min_confidence)
51
52     print(f"\nFrequent Itemsets for {db_name}:")
53     print(freq_itemsets)
54
55     print(f"\nAssociation Rules for {db_name}:")
56     print(rules)
57
58     print(f"\nApriori execution time for {db_name}: {apriori_time:.6f} seconds")
59
60     return apriori_time
61
62 # Define the databases and apply Apriori with predefined thresholds
63 databases = ['TransactionalDB1', 'TransactionalDB2', 'TransactionalDB3', 'TransactionalDB4', 'TransactionalDB5']
64
65 min_support = 0.1 # Predefined support value
66 min_confidence = 0.6 # Predefined confidence value
67
68 # Loop through each database and evaluate Apriori
69 for db_name in databases:
70     apriori_time = evaluate_apriori(db_name, min_support=min_support, min_confidence=min_confidence)
71

```


Results

The Apriori algorithm was applied to five transactional databases, producing frequent itemsets and association rules for each dataset. Below is a summary of key findings from each database:

TransactionalDB1:

Frequent itemsets and association rules in TransactionalDB1 showed strong associations between various products:

- Frequent itemsets:
 - Apples (support = 0.15), Cheese (support = 0.20), Rice (support = 0.20).
 - Frequent pairs include Apples and Bananas (support = 0.10), Rice and Potatoes (support = 0.15).
- Association rules:
 - Apples → Bananas (confidence = 66.67%, lift = 6.67) and Bananas → Apples (confidence = 100%, lift = 6.67).
 - Rice → Potatoes (confidence = 75%, lift = 5).
 - Diapers → Baby Wipes (confidence = 100%, lift = 10), indicating a strong correlation between these items.

Execution time for TransactionalDB1: 0.003992 seconds.

TransactionalDB2:

Results from TransactionalDB2 revealed slightly different patterns:

- Frequent itemsets:
 - Butter (support = 0.20), Fish (support = 0.20), Oranges (support = 0.25).
 - Frequent pairs include Rice and Potatoes (support = 0.10), Apples and Oranges (support = 0.10).
- Association rules:
 - Apples → Oranges (confidence = 100%, lift = 4), and Oranges → Apples (confidence = 60%).
 - Diapers → Baby Wipes (confidence = 100%, lift = 10).
 - Cereal → Milk (confidence = 100%, lift = 6.67).

Execution time for TransactionalDB2: 0.003116 seconds.

TransactionalDB3:

In TransactionalDB3, more diverse frequent itemsets were observed, including higher support values for certain items:

- Frequent itemsets:
 - Apples (support = 0.15), Cereal (support = 0.20), Rice (support = 0.25).
 - Frequent pairs include Bread and Milk (support = 0.10), Rice and Cheese (support = 0.10).
- Association rules:
 - Apples → Bananas (confidence = 66.67%, lift = 4.44), and Bananas → Apples (confidence = 66.67%).
 - Bread → Milk (confidence = 66.67%, lift = 6.67), and Milk → Bread (confidence = 100%, lift = 6.67).
 - Diapers → Baby Wipes (confidence = 100%, lift = 10).

Execution time for TransactionalDB3: 0.001990 seconds.

TransactionalDB4:

TransactionalDB4 exhibited some overlap with the previous datasets, along with unique combinations of itemsets:

- Frequent itemsets:
 - Apples (support = 0.15), Oranges (support = 0.25), Potatoes (support = 0.15).
 - Frequent pairs include Apples and Oranges (support = 0.15), Butter and Onions (support = 0.10).
- Association rules:
 - Apples → Oranges (confidence = 100%, lift = 4), and Oranges → Apples (confidence = 60%).
 - Bananas → Oranges (confidence = 100%, lift = 4), and Oranges → Bananas (confidence = 100%).
 - Laundry Detergent → Soap (confidence = 66.67%, lift = 4.44).

Execution time for TransactionalDB4: 0.002985 seconds.

TransactionalDB5:

Lastly, TransactionalDB5 provided insights into some less frequent but notable product pairings:

- Frequent itemsets:
 - Bread (support = 0.20), Cheese (support = 0.20), Oranges (support = 0.30).
 - Frequent pairs include Bread and Cheese (support = 0.20), Apples and Bananas (support = 0.10).
- Association rules:

- Apples → Bananas (confidence = 66.67%, lift = 6.67).
- Apples → Oranges (confidence = 100%, lift = 3.33), and Oranges → Apples (confidence = 33.33%).
- Bread → Cheese and Cheese → Bread, both with 100% confidence and a lift of 5.

Execution time for TransactionalDB5: 0.002991 seconds.

Overall, the Apriori algorithm identified a variety of frequent itemsets and association rules that reflect typical consumer behavior in these transactional datasets. The execution times for all databases were highly efficient, with an average time of approximately 0.002 seconds, demonstrating the scalability of the algorithm even with multiple databases.

Conclusion

The Apriori algorithm was successfully implemented to analyze five transactional databases, identifying frequent itemsets and deriving association rules for various products. The results of this analysis highlight several key insights into consumer purchasing behaviors and product associations, which can be leveraged in multiple ways.

The Apriori algorithm efficiently identified product combinations that frequently co-occur, such as Apples and Bananas, Rice and Potatoes, and Diapers and Baby Wipes. These associations suggest possible opportunities for cross-promotion and bundling strategies. Additionally, products like Oranges, Cheese, and Bread emerged as consistent frequent items, indicating their central role in multiple transactions across the databases.

By setting minimum support and confidence thresholds, the algorithm was able to balance performance with accuracy, generating meaningful rules without overfitting to the specific dataset characteristics. The average execution time of around 0.002 seconds demonstrates the algorithm's scalability and efficiency, making it a practical tool for real-time market basket analysis.

Furthermore, the analysis showed that the relationships between certain product pairs, such as **Diapers → Baby Wipes** and **Apples → Oranges**, were extremely strong, with confidence levels reaching 100% in many cases. These results indicate clear patterns in consumer behavior that businesses can exploit for improving customer satisfaction and driving sales through targeted promotions.

Key takeaways from this section include:

- **Efficiency:** The Apriori algorithm performed efficiently even across multiple datasets, with quick execution times that would be scalable to larger datasets.
- **Flexibility:** The algorithm's parameterization (minimum support and confidence thresholds) allows for fine-tuning based on the business context, making it adaptable for various domains beyond retail.

```

File Edit Selection View Go Run Terminal Help
Processing TransactionalDB1 with Apriori algorithm...

Frequent Itemsets for TransactionalDB1:
support
0 0.15 (Apples)
1 0.10 (Baby Wipes)
2 0.10 (Bananas)
3 0.15 (Bread)
4 0.15 (Butter)
5 0.10 (Cereal)
6 0.20 (Cheese)
7 0.15 (Diapers)
8 0.20 (Fish)
9 0.10 (Garlic)
10 0.10 (Milk)
11 0.15 (Onions)
12 0.20 (Oranges)
13 0.15 (Potatoes)
14 0.20 (Rice)
15 0.10 (Shampoo)
16 0.15 (Shaving Cream)
17 0.15 (Soup)
18 0.15 (Toilet Paper)
19 0.10 (Apples, Bananas)
20 0.10 (Diapers, Baby Wipes)
21 0.10 (Bread, Cheese)
22 0.10 (Fish, Rice)
23 0.10 (Garlic, Oranges)
24 0.10 (Potatoes, Onions)
25 0.10 (Rice, Onions)
26 0.15 (Rice, Potatoes)
27 0.10 (Shaving Cream, Soap)
28 0.10 (Toilet Paper, Shaving Cream)
29 0.10 (Rice, Potatoes, Onions)

Association Rules for TransactionalDB1:
antecedents consequents antecedent support consequent support support confidence lift leverage conviction zhangs_metric
0 (Apples) (Bananas) 0.15 0.10 0.10 0.66667 0.66667 0.8050 2.70 1.000000
1 (Bananas) (Apples) 0.10 0.15 0.10 1.000000 0.66667 0.8050 inf 0.944444
2 (Diapers) (Baby Wipes) 0.15 0.10 0.10 0.66667 0.66667 0.8050 2.70 1.000000
3 (Baby Wipes) (Diapers) 0.10 0.15 0.10 1.000000 0.66667 0.8050 inf 0.944444
4 (Bread) (Cheese) 0.15 0.20 0.10 0.66667 3.33333 0.8700 2.40 0.823529
5 (Garlic) (Oranges) 0.10 0.20 0.10 1.000000 5.000000 0.8000 inf 0.888889
6 (Potatoes) (Onions) 0.15 0.15 0.10 0.66667 4.44444 0.8775 2.55 0.911765
7 (Onions) (Potatoes) 0.15 0.15 0.10 0.66667 4.44444 0.8775 2.55 0.911765
8 (Onions) (Rice) 0.15 0.20 0.10 0.66667 3.33333 0.8700 2.40 0.823529
9 (Rice) (Potatoes) 0.20 0.15 0.15 0.750000 5.000000 0.1200 3.40 1.000000
10 (Potatoes) (Rice) 0.15 0.20 0.15 1.000000 5.000000 0.1200 inf 0.941176
11 (Shaving Cream) (Soap) 0.15 0.15 0.10 0.66667 4.44444 0.8775 2.55 0.911765
12 (Soap) (Shaving Cream) 0.15 0.15 0.10 0.66667 4.44444 0.8775 2.55 0.911765
13 (Toilet Paper) (Shaving Cream) 0.15 0.15 0.10 0.66667 4.44444 0.8775 2.55 0.911765
14 (Shaving Cream) (Toilet Paper) 0.15 0.15 0.10 0.66667 4.44444 0.8775 2.55 0.911765
15 (Rice, Potatoes) (Onions) 0.15 0.15 0.10 0.66667 4.44444 0.8775 2.55 0.911765
16 (Rice, Onions) (Potatoes) 0.10 0.15 0.10 1.000000 0.66667 0.8050 inf 0.944444
17 (Potatoes, Onions) (Rice) 0.10 0.20 0.10 1.000000 5.000000 0.8000 inf 0.888889

La 71, Col 1 Spaces 2 UTF-8 CRLF Python 3.11.8 (Python11) conda

```

```

File Edit Selection View Go Run Terminal Help
Apriori execution time for TransactionalDB1: 0.003992 seconds
Processing TransactionalDB2 with Apriori algorithm...

Frequent Itemsets for TransactionalDB2:
support
0 0.10 (Apples)
1 0.10 (Baby Wipes)
2 0.10 (Bananas)
3 0.10 (Bread)
4 0.20 (Butter)
5 0.10 (Cereal)
6 0.20 (Cheese)
7 0.10 (Diapers)
8 0.20 (Fish)
9 0.10 (Garlic)
10 0.10 (Laundry Detergent)
11 0.15 (Milk)
12 0.10 (Onions)
13 0.25 (Oranges)
14 0.10 (Paper Towels)
15 0.15 (Potatoes)
16 0.15 (Rice)
17 0.10 (Shampoo)
18 0.15 (Shaving Cream)
19 0.10 (Soup)
20 0.15 (Toilet Paper)
21 0.10 (Apples, Oranges)
22 0.10 (Diapers, Baby Wipes)
23 0.10 (Butter, Cheese)
24 0.10 (Fish, Butter)
25 0.10 (Cereal, Milk)
26 0.10 (Fish, Cheese)
27 0.10 (Garlic, Oranges)
28 0.10 (Potatoes, Onions)
29 0.10 (Rice, Onions)
30 0.10 (Toilet Paper, Paper Towels)
31 0.10 (Rice, Potatoes)
32 0.10 (Toilet Paper, Shaving Cream)
33 0.10 (Rice, Potatoes, Onions)

Association Rules for TransactionalDB2:
antecedents consequents antecedent support consequent support support confidence lift leverage conviction zhangs_metric
0 (Apples) (Oranges) 0.10 0.25 0.1 1.000000 4.000000 0.0750 inf 0.833333

La 71, Col 1 Spaces 2 UTF-8 CRLF Python 3.11.8 (Python11) conda

```

```

File Edit Selection View Go Run Terminal Help
← → Search

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

0 antecedents consequents antecedent support consequent support support confidence lift leverage conviction zhangs_metric
1 (Apples) (Bananas) 0.10 0.25 0.1 1.000000 4.000000 0.0750 inf 0.833333
2 (Diapers) (Baby Wipes) 0.10 0.10 0.1 1.000000 10.000000 0.0900 inf 1.000000
3 (Cereal) (Milk) 0.10 0.15 0.1 1.000000 6.666667 0.0850 inf 0.944444
4 (Milk) (Cereal) 0.15 0.10 0.1 0.666667 6.666667 0.0850 2.70 1.000000
5 (Garlic) (Onions) 0.10 0.25 0.1 1.000000 4.000000 0.0750 inf 0.833333
6 (Potatoes) (Onions) 0.15 0.10 0.1 0.666667 6.666667 0.0850 2.70 1.000000
7 (Onions) (Potatoes) 0.10 0.15 0.1 1.000000 6.666667 0.0850 inf 0.944444
8 (Rice) (Onions) 0.15 0.10 0.1 0.666667 6.666667 0.0850 2.70 1.000000
9 (Onions) (Rice) 0.10 0.15 0.1 1.000000 6.666667 0.0850 inf 0.944444
10 (Toilet Paper) (Paper Towels) 0.15 0.10 0.1 0.666667 6.666667 0.0850 2.70 1.000000
11 (Paper Towels) (Toilet Paper) 0.10 0.15 0.1 1.000000 6.666667 0.0850 inf 0.944444
12 (Rice) (Potatoes) 0.15 0.15 0.1 0.666667 4.444444 0.0775 2.55 0.911705
13 (Potatoes) (Rice) 0.15 0.15 0.1 0.666667 4.444444 0.0775 2.55 0.911705
14 (Toilet Paper) (Shaving Cream) 0.15 0.15 0.1 0.666667 4.444444 0.0775 2.55 0.911705
15 (Shaving Cream) (Toilet Paper) 0.15 0.15 0.1 0.666667 4.444444 0.0775 2.55 0.911705
16 (Rice, Potatoes) (Onions) 0.10 0.10 0.1 1.000000 10.000000 0.0900 inf 1.000000
17 (Rice, Onions) (Potatoes) 0.10 0.15 0.1 1.000000 6.666667 0.0850 inf 0.944444
18 (Potatoes, Onions) (Rice) 0.10 0.15 0.1 1.000000 6.666667 0.0850 inf 0.944444
19 (Rice) (Potatoes, Onions) 0.15 0.10 0.1 0.666667 6.666667 0.0850 2.70 1.000000
20 (Potatoes) (Rice, Onions) 0.15 0.10 0.1 0.666667 6.666667 0.0850 2.70 1.000000
21 (Onions) (Rice, Potatoes) 0.10 0.10 0.1 1.000000 10.000000 0.0900 inf 1.000000

Apriori execution time for TransactionalDB2: 0.003116 seconds
Processing TransactionalDB3 with Apriori algorithm...

Frequent Itemsets for TransactionalDB3:
support itemsets
0 0.15 (Apples)
1 0.10 (Baby Wipes)
2 0.15 (Bananas)
3 0.15 (Bread)
4 0.15 (Butter)
5 0.20 (Cereal)
6 0.20 (Cheese)
7 0.10 (Diapers)
8 0.15 (Fish)
9 0.15 (Garlic)
10 0.10 (Milk)
11 0.10 (Onions)
12 0.20 (Oranges)
13 0.15 (Potatoes)
14 0.25 (Rice)
15 0.15 (Shampoo)
16 0.10 (Shaving Cream)
17 0.10 (Soup)
18 0.10 (Toilet Paper)
19 0.10 (Apples, Bananas)
20 0.10 (Diapers, Baby Wipes)
21 0.10 (Bread, Milk)
22 0.10 (Cereal, Cheese)
23 0.10 (Rice, Cereal)
24 0.10 (Rice, Cheese)

La 71, Col 1 Spaces 2 UTF-8 CRLF Python 3.11.8 (Python11) conda

```

```

File Edit Selection View Go Run Terminal Help
← → Search

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

23 0.10 (Rice, Cereal)
24 0.10 (Rice, Cheese)
25 0.10 (Fish, Garlic)

Association Rules for TransactionalDB3:
antecedents consequents antecedent support consequent support support confidence lift leverage conviction zhangs_metric
0 (Apples) (Bananas) 0.15 0.15 0.1 0.666667 4.444444 0.0775 2.55 0.911705
1 (Bananas) (Apples) 0.15 0.15 0.1 0.666667 4.444444 0.0775 2.55 0.911705
2 (Diapers) (Baby Wipes) 0.10 0.10 0.1 1.000000 10.000000 0.0900 inf 1.000000
3 (Baby Wipes) (Diapers) 0.10 0.10 0.1 1.000000 10.000000 0.0900 inf 1.000000
4 (Bread) (Milk) 0.15 0.15 0.1 0.666667 6.666667 0.0850 2.70 1.000000
5 (Milk) (Bread) 0.10 0.15 0.1 1.000000 6.666667 0.0850 inf 0.944444
6 (Fish) (Garlic) 0.15 0.15 0.1 0.666667 4.444444 0.0775 2.55 0.911705
7 (Garlic) (Fish) 0.15 0.15 0.1 0.666667 4.444444 0.0775 2.55 0.911705

Apriori execution time for TransactionalDB3: 0.001998 seconds
Processing TransactionalDB4 with Apriori algorithm...

Frequent Itemsets for TransactionalDB4:
support itemsets
0 0.15 (Apples)
1 0.10 (Baby Wipes)
2 0.10 (Bananas)
3 0.15 (Bread)
4 0.15 (Butter)
5 0.10 (Cereal)
6 0.15 (Cheese)
7 0.10 (Diapers)
8 0.15 (Fish)
9 0.10 (Garlic)
10 0.15 (Laundry Detergent)
11 0.10 (Milk)
12 0.15 (Onions)
13 0.25 (Oranges)
14 0.15 (Potatoes)
15 0.20 (Rice)
16 0.10 (Shampoo)
17 0.10 (Shaving Cream)
18 0.15 (Soup)
19 0.15 (Toilet Paper)
20 0.10 (Apples, Bananas)
21 0.15 (Apples, Oranges)
22 0.10 (Diapers, Baby Wipes)
23 0.10 (Oranges, Bananas)
24 0.10 (Bread, Cheese)
25 0.10 (Butter, Onions)
26 0.10 (Laundry Detergent, Soap)
27 0.10 (Rice, Milk)
28 0.10 (Potatoes, Onions)
29 0.10 (Toilet Paper, Shaving Cream)
30 0.10 (Apples, Oranges, Bananas)

Association Rules for TransactionalDB4:
antecedents consequents antecedent support consequent support support confidence lift leverage conviction zhangs_metric

La 71, Col 1 Spaces 2 UTF-8 CRLF Python 3.11.8 (Python11) conda

```

```

File Edit Selection View Go Run Terminal Help
← → Search

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Association Rules for TransactionalDB4:
antecedents consequents antecedent support consequent support support confidence lift leverage conviction zhangs_metric
0 (Apples) (Bananas) 0.15 0.10 0.10 0.666667 6.666667 0.0050 2.700 1.000000
1 (Bananas) (Apples) 0.10 0.15 0.10 1.000000 6.666667 0.0050 inf 0.944444
2 (Apples) (Oranges) 0.15 0.25 0.15 1.000000 4.000000 0.1125 inf 0.802253
3 (Oranges) (Apples) 0.25 0.15 0.15 0.000000 4.000000 0.1125 2.125 1.000000
4 (Diapers) (Baby Wipes) 0.10 0.10 0.10 1.000000 10.000000 0.0000 inf 1.000000
5 (Baby Wipes) (Diapers) 0.10 0.10 0.10 1.000000 10.000000 0.0000 inf 1.000000
6 (Bananas) (Oranges) 0.10 0.25 0.10 1.000000 4.000000 0.0750 inf 0.813333
7 (Bread) (Cheese) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.550 0.911765
8 (Cheese) (Bread) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.550 0.911765
9 (Butter) (Onions) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.550 0.911765
10 (Onions) (Butter) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.550 0.911765
11 (Laundry Detergent) (Soap) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.550 0.911765
12 (Soap) (Laundry Detergent) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.550 0.911765
13 (Milk) (Rice) 0.10 0.20 0.10 1.000000 5.000000 0.0000 inf 0.800000
14 (Potatoes) (Onions) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.550 0.911765
15 (Onions) (Potatoes) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.550 0.911765
16 (Toilet Paper) (Shaving Cream) 0.15 0.10 0.10 0.666667 6.666667 0.0050 2.700 1.000000
17 (Shaving Cream) (Toilet Paper) 0.10 0.15 0.10 1.000000 6.666667 0.0050 inf 0.944444
18 (Apples, Oranges) (Bananas) 0.15 0.10 0.10 0.666667 6.666667 0.0050 2.700 1.000000
19 (Apples, Bananas) (Oranges) 0.10 0.25 0.10 1.000000 4.000000 0.0750 inf 0.813333
20 (Oranges, Bananas) (Apples) 0.10 0.15 0.10 1.000000 6.666667 0.0050 inf 0.944444
21 (Apples) (Oranges, Bananas) 0.15 0.10 0.10 0.666667 6.666667 0.0050 2.700 1.000000
22 (Bananas) (Apples, Oranges) 0.10 0.15 0.10 1.000000 6.666667 0.0050 inf 0.944444

Apriori execution time for TransactionalDB4: 0.002985 seconds

Processing TransactionalDB5 with Apriori algorithm...

Frequent Itemsets for TransactionalDB5:
support itemsets
0 0.15 (Apples)
1 0.10 (Baby Wipes)
2 0.10 (Bananas)
3 0.20 (Bread)
4 0.15 (Butter)
5 0.10 (Cereal)
6 0.20 (Cheese)
7 0.10 (Diapers)
8 0.15 (Fish)
9 0.15 (Garlic)
10 0.10 (Lettuce)
11 0.15 (Onions)
12 0.20 (Oranges)
13 0.10 (Paper Towels)
14 0.15 (Potatoes)
15 0.15 (Rice)
16 0.10 (Shampoo)
17 0.15 (Soup)
18 0.10 (Toilet Paper)
19 0.10 (Apples, Bananas)
20 0.15 (Apples, Oranges)
21 0.10 (Diapers, Baby Wipes)
22 0.10 (Oranges, Bananas)

La 71, Col 1 Spaces 2 UTF-8 CR LF Python 3.11.8 (Python11) conda

```

```

File Edit Selection View Go Run Terminal Help
← → Search

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

1 0.10 (Baby Wipes)
2 0.10 (Bananas)
3 0.20 (Bread)
4 0.15 (Butter)
5 0.10 (Cereal)
6 0.20 (Cheese)
7 0.10 (Diapers)
8 0.15 (Fish)
9 0.15 (Garlic)
10 0.10 (Lettuce)
11 0.15 (Onions)
12 0.20 (Oranges)
13 0.10 (Paper Towels)
14 0.15 (Potatoes)
15 0.15 (Rice)
16 0.10 (Shampoo)
17 0.15 (Soup)
18 0.10 (Toilet Paper)
19 0.10 (Apples, Bananas)
20 0.15 (Apples, Oranges)
21 0.10 (Diapers, Baby Wipes)
22 0.10 (Oranges, Bananas)
23 0.20 (Bread, Cheese)
24 0.10 (Garlic, Onions)
25 0.10 (Lettuce, Oranges)
26 0.10 (Rice, Onions)
27 0.10 (Soup, Paper Towels)
28 0.10 (Apples, Oranges, Bananas)

Association Rules for TransactionalDB5:
antecedents consequents antecedent support consequent support support confidence lift leverage conviction zhangs_metric
0 (Apples) (Bananas) 0.15 0.10 0.10 0.666667 6.666667 0.0050 2.70 1.000000
1 (Bananas) (Apples) 0.10 0.15 0.10 1.000000 6.666667 0.0050 inf 0.944444
2 (Apples) (Oranges) 0.15 0.30 0.15 1.000000 3.333333 0.1050 inf 0.823529
3 (Diapers) (Baby Wipes) 0.10 0.10 0.10 1.000000 10.000000 0.0000 inf 1.000000
4 (Baby Wipes) (Diapers) 0.10 0.10 0.10 1.000000 10.000000 0.0000 inf 1.000000
5 (Bananas) (Oranges) 0.10 0.30 0.10 1.000000 3.333333 0.0700 inf 0.777778
6 (Bread) (Cheese) 0.20 0.20 0.20 1.000000 5.000000 0.1000 inf 1.000000
7 (Cheese) (Bread) 0.20 0.20 0.20 1.000000 5.000000 0.1000 inf 1.000000
8 (Garlic) (Onions) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.55 0.911765
9 (Onions) (Garlic) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.55 0.911765
10 (Lettuce) (Oranges) 0.10 0.30 0.10 1.000000 3.333333 0.0700 inf 0.777778
11 (Rice) (Onions) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.55 0.911765
12 (Onions) (Rice) 0.15 0.15 0.10 0.666667 4.444444 0.0775 2.55 0.911765
13 (Soup) (Paper Towels) 0.15 0.10 0.10 0.666667 6.666667 0.0050 2.70 1.000000
14 (Paper Towels) (Soup) 0.10 0.15 0.10 1.000000 6.666667 0.0050 inf 0.944444
15 (Apples, Oranges) (Bananas) 0.15 0.10 0.10 0.666667 6.666667 0.0050 2.70 1.000000
16 (Apples, Bananas) (Oranges) 0.10 0.30 0.10 1.000000 3.333333 0.0700 inf 0.777778
17 (Oranges, Bananas) (Apples) 0.10 0.15 0.10 1.000000 6.666667 0.0050 inf 0.944444
18 (Apples) (Oranges, Bananas) 0.15 0.10 0.10 0.666667 6.666667 0.0050 2.70 1.000000
19 (Bananas) (Apples, Oranges) 0.10 0.15 0.10 1.000000 6.666667 0.0050 inf 0.944444

Apriori execution time for TransactionalDB5: 0.002991 seconds
PS C:\Users\labeng>

La 71, Col 1 Spaces 2 UTF-8 CR LF Python 3.11.8 (Python11) conda

```

Brute Force Algorithm Implementation

The brute force algorithm systematically examines all possible itemsets to find frequent ones that meet the minimum support threshold. Unlike the Apriori algorithm, which efficiently reduces the search space, the brute force approach evaluates every possible combination of items, making it a more computationally intensive method.

Fetching Transactions

The brute force algorithm begins by fetching transactional data from a database, just like the Apriori algorithm. The `fetch_transactions()` function is used to connect to the SQL Server and retrieve transaction records.

```

7  # Function to fetch transactions from a specific database
8  def fetch_transactions(db_name):
9      try:
10         connection_string = f'mssql+pyodbc://localhost/{db_name}?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes'
11         engine = sa.create_engine(connection_string)
12         df = pd.read_sql("SELECT t.TransactionID, p.ProductName FROM Transactions t JOIN Products p ON t.ProductID = p.ProductID", engine)
13         return df
14     except Exception as e:
15         print(f"Error reading data from {db_name}: ", e)
16         return None
17

```

Preprocessing the Data

The next step is to preprocess the transaction data into a basket format, where each row represents a transaction, and each column represents a product. The value 1 indicates that the product was purchased in that transaction, while 0 indicates that it was not.

```

18 # Preprocessing function to convert transactions into a format suitable for itemset generation
19 def preprocess_transactions(df):
20     basket = df.groupby(['TransactionID', 'ProductName'])['ProductName'].count().unstack().reset_index().fillna(0).set_index('TransactionID')
21     basket = basket.map(lambda x: 1 if x > 0 else 0)
22     return basket
23

```

Generating Itemsets

The brute force approach generates all possible itemsets of size 1 to k (limited to 5-itemsets in this implementation). This is done using the `generate_itemsets()` function, which leverages Python's `itertools.combinations()` to create combinations of products.

```

24 # Generate all possible itemsets of size 1 to k
25 def generate_itemsets(items, k):
26     itemsets = []
27     max_k = 5 # Limit to 5-itemsets
28     for size in range(1, min(k, max_k)+1):
29         itemsets.extend(combinations(items, size))
30     return itemsets
31

```

In contrast to the Apriori algorithm, which prunes non-frequent itemsets early on, the brute force method generates all itemsets regardless of their frequency. This increases the computational complexity, especially as the number of items grows.

Checking Support for Itemsets

Once all itemsets are generated, the algorithm checks the support of each itemset by counting how many transactions contain all items in the set. If the support meets or exceeds the minimum threshold, the itemset is considered frequent and added to the list.

```

33 # Check the support of each itemset
34 def get_frequent_itemsets(itemsets, basket, min_support):
35     frequent_itemsets = []
36     num_transactions = len(basket)
37
38     for idx, itemset in enumerate(itemsets):
39         # Check how many transactions contain all items in the itemset
40         support_count = (basket[list(itemset)].sum(axis=1) == len(itemset)).sum()
41         support = support_count / num_transactions
42
43         if support >= min_support:
44             frequent_itemsets.append((itemset, support))
45
46     return frequent_itemsets
47

```

Applying the Brute Force Algorithm

The `brute_force_apriori()` function orchestrates the brute force search by repeatedly generating itemsets of increasing size (up to `max_k`) and calculating their support. The function terminates when no frequent itemsets are found for a given size, as further iterations would not yield frequent itemsets.

```

48 # Brute force method to find frequent itemsets
49 def brute_force_apriori(basket, min_support, max_k=3): # Pass max_k as a parameter
50     items = basket.columns
51     k = 1
52     all_frequent_itemsets = []
53
54     while k <= max_k: # Stop when k exceeds max_k
55         print(f"\nGenerating {k}-itemsets...")
56         itemsets = generate_itemsets(items, k)
57         frequent_itemsets = get_frequent_itemsets(itemsets, basket, min_support)
58
59         if not frequent_itemsets:
60             print(f"No frequent {k}-itemsets found. Stopping.")
61             break # Terminate when no frequent itemsets are found
62
63         all_frequent_itemsets.extend(frequent_itemsets)
64         k += 1 # Move to the next size
65
66     # Convert to DataFrame for use with association_rules function
67     frequent_itemsets_df = pd.DataFrame(all_frequent_itemsets, columns=['itemsets', 'support'])
68
69     return frequent_itemsets_df
70

```


Generating Association Rules

Once the frequent itemsets are found, the `generate_association_rules()` function is applied to derive association rules from the itemsets. This function uses the `mlxtend` library for consistency with the Apriori algorithm.

```

72 # Function to generate association rules from frequent itemsets
73 def generate_association_rules(frequent_itemsets_df, min_confidence=0.6):
74     if frequent_itemsets_df.empty:
75         print("No frequent itemsets found to generate association rules.")
76         return pd.DataFrame()
77
78     # Using the mlxtend library's association_rules function for consistency with Apriori
79     rules = association_rules(frequent_itemsets_df, metric="confidence", min_threshold=min_confidence)
80     return rules
81

```

Timing the Brute Force Algorithm

Finally, the `time_brute_force_with_rules()` function is used to time the execution of the brute force algorithm, measure the performance, and print the resulting frequent itemsets and association rules for each database.

```

82 # Time the brute force algorithm and print association rules
83 def time_brute_force_with_rules(db_name, min_support, min_confidence):
84     df = fetch_transactions(db_name)
85     if df is not None:
86         basket = preprocess_transactions(df)
87         print(f"\nBasket for {db_name}:")
88         print(basket)
89         start_time = time.time()
90         frequent_itemsets_df = brute_force_apriori(basket, min_support)
91         end_time = time.time()
92
93         print(f"\nBrute Force Frequent Itemsets for {db_name}:")
94         print(frequent_itemsets_df)
95
96         # Generate association rules
97         rules = generate_association_rules(frequent_itemsets_df, min_confidence)
98
99         print(f"\nAssociation Rules for {db_name}:")
100        print(rules)
101
102        print(f"\nBrute force time for {db_name}: {end_time - start_time} seconds")
103        return end_time - start_time, rules
104
105 # Apply brute force method to all databases and print rules
106 databases = ['TransactionalDB1', 'TransactionalDB2', 'TransactionalDB3', 'TransactionalDB4', 'TransactionalDB5']
107 min_support = 0.1 # Adjust this value as necessary
108 min_confidence = 0.6 # Adjust confidence threshold if needed
109
110 for db_name in databases:
111     print(f"\nProcessing {db_name} with brute force method...")
112     brute_force_time, rules = time_brute_force_with_rules(db_name, min_support, min_confidence)
113

```

Results

The brute force method was applied to all five transactional databases, and the results include the frequent itemsets and the association rules derived from these itemsets. Below is a summary of key findings from each database:

TransactionalDB1 Results

- Frequent Itemsets: A total of 78 frequent itemsets were identified, ranging from single items (e.g., Apples, Bananas, Bread) to more complex itemsets like (Onions, Rice) and (Shaving Cream, Soap).
- Association Rules: 20 association rules were generated. Examples include:
 - (Apples → Bananas) with a confidence of 0.67 and a lift of 6.67.
 - (Baby Wipes → Diapers) and (Diapers → Baby Wipes) both with a confidence of 1.0 and a lift of 6.67.
- Execution Time: The brute force method took 1.03 seconds to complete.

TransactionalDB2 Results

- Frequent Itemsets: 88 frequent itemsets were identified. Popular combinations include (Apples, Oranges), (Bread, Cheese), and (Onions, Potatoes).
- Association Rules: 22 rules were derived, such as:
 - (Apples → Oranges) with a confidence of 1.0 and a lift of 4.0.
 - (Cereal → Milk) and (Milk → Cereal) with confidence of 1.0 and lift of 6.67.
- Execution Time: The process took 1.19 seconds.

TransactionalDB3 Results

- Frequent Itemsets: 71 frequent itemsets were found. Key itemsets include (Apples, Bananas), (Fish, Garlic), and (Cereal, Cheese).
- Association Rules: 8 rules were generated. Some of the top rules include:
 - (Bread → Milk) with a confidence of 0.67 and a lift of 6.67.
 - (Fish → Garlic) with a confidence of 0.67 and a lift of 4.44.
- Execution Time: The algorithm completed in 1.07 seconds.

TransactionalDB4 Results

- Frequent Itemsets: 81 frequent itemsets were identified, including combinations like (Laundry Detergent, Soap) and (Onions, Potatoes).
- Association Rules: 23 rules were extracted. Examples include:
 - (Apples → Bananas) with a confidence of 0.67 and a lift of 6.67.
 - (Baby Wipes → Diapers) with a confidence of 1.0 and a lift of 10.0.
- Execution Time: The brute force process took 1.20 seconds.

TransactionalDB5 Results

- **Frequent Itemsets:** 76 frequent itemsets were identified, with combinations like (Bread, Cheese) and (Garlic, Onions).
- **Association Rules:** 20 rules were generated. Key examples include:
 - (Bread → Cheese) with a confidence of 1.0 and a lift of 5.0.
 - (Bananas → Oranges) with a confidence of 1.0 and a lift of 3.33.
- **Execution Time:** The execution took 1.23 seconds.

The brute force algorithm effectively identified frequent itemsets and association rules across all five transactional databases. The average completion time for each database was approximately **1.14 seconds**, showing consistent and reasonable performance given the dataset size. The algorithm successfully uncovered strong relationships between products, such as **Baby Wipes and Diapers** and **Apples and Bananas**, with high confidence. While the brute force method performed well on these datasets, its exhaustive nature might limit its scalability to larger or more complex datasets.

Conclusion

The brute force algorithm was successfully applied to five transactional databases, identifying frequent itemsets and generating association rules for various product combinations. The results revealed meaningful insights into consumer behavior, uncovering patterns such as **Apples and Bananas**, **Diapers and Baby Wipes**, and **Onions and Potatoes**.

Despite being computationally exhaustive, the brute force method completed its analysis in an average of **1.14 seconds** per database. This demonstrates its practicality for relatively small datasets but highlights potential limitations in scalability as dataset sizes grow. The algorithm's ability to consistently find strong product relationships across multiple databases reaffirms its accuracy and reliability, although larger datasets may require more optimized approaches.

Key takeaways from this section include:

- **Accuracy:** The brute force algorithm consistently identified frequent itemsets and association rules with strong support and confidence metrics, highlighting its reliability in smaller datasets.
- **Performance:** While the execution times were manageable for these datasets, the algorithm's exhaustive nature may present challenges when scaling to larger datasets, where more efficient algorithms like Apriori may be preferred.

File Edit Selection View Go Run Terminal Help

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\aberg> C:\Users\aberg\anaconda3\envs\python11\python.exe "C:\Users\aberg\OneDrive\Desktop\NOIT\Data Mining\Midterm Project\Midterm Brute Force.py"

Processing TransactionalDBI with brute force method...

Basket for TransactionalDBI:

TransactionID	Apples	Baby Wipes	Bananas	Bread	Butter	Carrots	Cereal	Cheese	Diapers	Eggs	Fish	Garlic	Laundry Detergent	Lettuce	Milk	Onions	Oranges	Pasta	Potatoes	Rice	Shampoo	Shaving Cream	Soap	Toilet Paper
1	0	0	0	1	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0
2	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
3	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0
5	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	1	0	0	0	0	0	0	0
7	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
8	1	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0	0	0	0
10	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	1	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1
13	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0
14	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1
15	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	1	0	0	0	0
16	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
17	0	0	0	1	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
18	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	0	0	0
19	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0
20	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0

Generating 1-Itemsets...

Generating 2-Itemsets...

Generating 3-Itemsets...

Brute Force Frequent Itemsets for TransactionalDBI:

Itemsets support

0	(Apples,)	0.15
1	(Baby Wipes,)	0.10
2	(Bananas,)	0.10
3	(Bread,)	0.15
4	(Butter,)	0.15
...	...	...
73	(Onions, Rice)	0.10
76	(Potatoes, Rice)	0.15
75	(Shaving Cream, Soap)	0.10
76	(Shaving Cream, Toilet Paper)	0.10
77	(Onions, Potatoes, Rice)	0.10

[78 rows x 2 columns]

Association Rules for TransactionalDBI:

antecedent	consequent	antecedent support	consequent support	support confidence	lift	leverage	conviction	chazng_metric
0	(Bananas)	0.10	0.15	0.10	1.00			

[illegible]

[illegible]

Result Comparison

In this section, let's compare the performance and results of the **Apriori algorithm** and the **brute force method** applied to the same set of transactional databases. The goal is to highlight the key differences in execution time, scalability, accuracy, and overall practicality of each algorithm.

Execution Time

One of the most noticeable differences between the two algorithms is the execution time. The Apriori algorithm consistently outperformed the brute force method in terms of speed, completing its analysis in an average of **0.002 seconds** across the databases. In contrast, the brute force algorithm took an average of **1.14 seconds** per database. While both execution times are manageable for smaller datasets, the significant difference illustrates the efficiency of Apriori, particularly for larger datasets. This gap suggests that the brute force method, while accurate, may become impractical as dataset size increases.

Scalability

The Apriori algorithm is inherently more scalable due to its optimized approach of pruning non-frequent itemsets early in the process. This allows it to handle larger datasets more efficiently without needing to exhaustively check all possible item combinations. On the other hand, the brute force method generates all possible itemsets up to a certain size, leading to a rapid increase in computational cost as the number of items grows. This makes Apriori a more suitable choice for real-world applications, where databases tend to be large and computational resources are limited.

Accuracy and Results

Both algorithms produced similar sets of frequent itemsets and association rules, with strong relationships such as **Diapers → Baby Wipes**, **Apples → Oranges**, and **Onions → Potatoes** appearing consistently across the databases. The **confidence levels** and **support values** were also comparable, indicating that both methods are reliable in terms of the accuracy of the associations they generate.

However, the brute force method's exhaustive nature ensures that no possible frequent itemsets are missed, whereas Apriori's pruning technique could potentially overlook some rare but significant itemsets if the minimum support threshold is set too high. Thus, while Apriori is more efficient, the brute force method offers more exhaustive results when dealing with smaller datasets.

Practicality and Flexibility

The **Apriori algorithm** emerges as the more practical choice for most real-world applications due to its balance of speed and accuracy. Its flexibility in terms of adjusting **minimum support** and **confidence thresholds** makes it adaptable to a wide range of business contexts, from retail to healthcare and beyond. The brute force method, while highly accurate in smaller datasets, lacks the flexibility needed for larger-scale applications due to its high computational cost.

Summary of Key Differences

Aspect	Apriori Algorithm	Brute Force Algorithm
Execution Time	~0.002 seconds (average)	~1.14 seconds (average)
Scalability	High (handles larger datasets well)	Low (computational cost increases rapidly)
Accuracy	High (may miss rare itemsets)	Very High (exhaustive search of all itemsets)
Practicality	Suitable for large-scale applications	Suitable for small datasets only
Flexibility	Adjustable thresholds for support/confidence	Less flexible, limited by computation

Conclusion

The **Apriori algorithm** is the more efficient and scalable option, especially when working with larger datasets. It strikes a good balance between speed, accuracy, and practicality. The **brute force method**, while exhaustive and highly accurate in smaller datasets, quickly becomes impractical as the size of the data increases. Therefore, the choice between these two methods should be guided by the dataset size and the computational resources available.

Conclusion

This project explored the application of two key data mining algorithms—**Apriori** and **Brute Force**—to uncover frequent itemsets and generate association rules from multiple transactional databases. By doing so, the project aimed to identify patterns in customer purchasing behaviors that businesses can leverage for marketing, cross-selling, and inventory management.

The **Apriori algorithm** proved to be the more practical solution for large-scale applications. Its efficiency in generating frequent itemsets by pruning non-frequent combinations early made it the preferred method, especially as the size of the data increased. The **average execution time** of 0.002 seconds per database highlights its suitability for real-time analysis and scalability to larger datasets. Moreover, its flexibility with adjustable thresholds for support and confidence makes it adaptable to various business needs, allowing users to fine-tune the algorithm based on different market contexts. This algorithm successfully identified key product relationships, such as **Apples → Oranges** and **Diapers → Baby Wipes**, which can be used for targeted marketing or bundling strategies.

The **brute force method**, on the other hand, while highly accurate and exhaustive, was limited in its practicality for large datasets. Its exhaustive search for all possible itemsets, though valuable for smaller datasets, resulted in significantly higher computation times, with an average of **1.14 seconds per database**. This approach does ensure that no potentially frequent itemset is missed but comes at the cost of scalability and efficiency. Therefore, while useful for small-scale analysis, the brute force method struggles with larger databases where computational resources are a concern.

Through the comparison of these two algorithms, the project demonstrated the importance of balancing **efficiency** and **accuracy** when choosing a method for frequent itemset mining. The **Apriori algorithm** was able to provide accurate results in a fraction of the time required by the brute force method, making it the optimal choice for most real-world applications.

From a business perspective, this project highlighted the potential to extract valuable insights from transactional data. By identifying strong associations, such as **Diapers → Baby Wipes** or **Apples → Oranges**, businesses can develop more targeted promotions, optimize product placement, and enhance customer satisfaction by predicting purchase patterns.

Key Takeaways

1. **Algorithm Selection:** The choice of algorithm depends on the dataset size and computational resources available. For large datasets, the **Apriori algorithm** offers a balanced solution, whereas the **brute force method** is best suited for small datasets requiring exhaustive analysis.
2. **Efficiency and Scalability:** The **Apriori algorithm's** ability to prune non-frequent itemsets early makes it significantly more efficient and scalable than the brute force approach.
3. **Actionable Insights:** Frequent itemset mining revealed key associations between products, which businesses can use for strategic decision-making, including product bundling, cross-selling, and inventory management.

4. **Flexibility:** The **Apriori algorithm** offers flexibility through its adjustable support and confidence thresholds, allowing businesses to tailor the algorithm based on specific operational goals or industry requirements.

Final Thoughts

This project demonstrates the power of data mining techniques for uncovering hidden patterns within transactional data. The results of this analysis reinforce the value of frequent itemset mining in understanding consumer behavior and providing actionable insights for business decisions. As organizations continue to accumulate larger volumes of transactional data, efficient and scalable algorithms like **Apriori** will become the main tools for harnessing the potential of this data to drive smarter business decisions through data mining.

References

SQLAlchemy Authors. (2020). **SQLAlchemy Documentation**. Retrieved from <https://docs.sqlalchemy.org/en/14/>

Pandas Development Team. (2020). **Pandas: Python Data Analysis Library**. Retrieved from <https://pandas.pydata.org/>

Raschka, S. (2018). **MLxtend: A library for machine learning and data science extensions**. Retrieved from <https://rasbt.github.io/mlxtend/>

MDanish99 (2022). **Apriori**. Retrieved from <https://github.com/MDanish99/Apriori>

Noble, J. (2024). **Implementing the Apriori algorithm in Python on watsonx.ai**. Retrieved from <https://developer.ibm.com/tutorials/awb-implementing-apriori-algorithm-python/>

Stack Overflow Thread (2012-Current). **Python Brute Force algorithm [closed]**. Retrieved from <https://stackoverflow.com/questions/11747254/python-brute-force-algorithm>

ChatGPT (Used solely for the purpose of debugging, concept explanation, and editing). Retrieved from <https://chatgpt.com>

Source Code

Apriori Algorithm from Scratch – Bisma Khan

(<https://www.kaggle.com/code/bismakhan08/apriori-algorithm-from-scratch>)

```
import pandas as pd
```

```
import numpy as np
```

```
import math
```

In [2]:

```
transaction_df = pd.read_csv('../input/supermarket/GroceryStoreDataSet.csv')
```

```
transaction_df
```

Out[2]:

	MILK,BREAD,BISCUIT
0	BREAD,MILK,BISCUIT,CORNFLAKES
1	BREAD,TEA,BOURNVITA
2	JAM,MAGGI,BREAD,MILK
3	MAGGI,TEA,BISCUIT
4	BREAD,TEA,BOURNVITA
5	MAGGI,TEA,CORNFLAKES
6	MAGGI,BREAD,TEA,BISCUIT
7	JAM,MAGGI,BREAD,TEA
8	BREAD,MILK
9	COFFEE,COCK,BISCUIT,CORNFLAKES
10	COFFEE,COCK,BISCUIT,CORNFLAKES
11	COFFEE,SUGER,BOURNVITA
12	BREAD,COFFEE,COCK

	MILK,BREAD,BISCUIT
13	BREAD,SUGER,BISCUIT
14	COFFEE,SUGER,CORNFLAKES
15	BREAD,SUGER,BOURNVITA
16	BREAD,COFFEE,SUGER
17	BREAD,COFFEE,SUGER
18	TEA,MILK,COFFEE,CORNFLAKES

In [3]:

```
transaction_df.index.rename('TID', inplace=True)
```

```
transaction_df.rename(columns={'MILK,BREAD,BISCUIT' : 'item_list'}, inplace=True)
```

In [4]:

```
trans_df = transaction_df.item_list.str.split(',')
```

```
trans_df
```

Out[4]:

TID

```

0  [BREAD, MILK, BISCUIT, CORNFLAKES]
1      [BREAD, TEA, BOURNVITA]
2      [JAM, MAGGI, BREAD, MILK]
3      [MAGGI, TEA, BISCUIT]
4      [BREAD, TEA, BOURNVITA]
5      [MAGGI, TEA, CORNFLAKES]
6      [MAGGI, BREAD, TEA, BISCUIT]
7      [JAM, MAGGI, BREAD, TEA]
8      [BREAD, MILK]
9  [COFFEE, COCK, BISCUIT, CORNFLAKES]
10 [COFFEE, COCK, BISCUIT, CORNFLAKES]
11 [COFFEE, SUGER, BOURNVITA]
```

```

12      [BREAD, COFFEE, COCK]
13      [BREAD, SUGER, BISCUIT]
14      [COFFEE, SUGER, CORNFLAKES]
15      [BREAD, SUGER, BOURNVITA]
16      [BREAD, COFFEE, SUGER]
17      [BREAD, COFFEE, SUGER]
18      [TEA, MILK, COFFEE, CORNFLAKES]

```

Name: item_list, dtype: object

In [5]:

```
def prune(data,supp):
```

```

    df = data[data.supp_count >= supp]
    return df

```

```
def count_itemset(transaction_df, itemsets):
```

```

    count_item = {}
    for item_set in itemsets:
        set_A = set(item_set)
        for row in trans_df:
            set_B = set(row)

            if set_B.intersection(set_A) == set_A:
                if item_set in count_item.keys():
                    count_item[item_set] += 1

            else:
                count_item[item_set] = 1

```

```

data = pd.DataFrame()
data['item_sets'] = count_item.keys()
data['supp_count'] = count_item.values()

```

```

return data

```

```

def count_item(trans_items):

```

```

    count_ind_item = {}
    for row in trans_items:
        for i in range(len(row)):
            if row[i] in count_ind_item.keys():
                count_ind_item[row[i]] += 1
            else:
                count_ind_item[row[i]] = 1

```

```

data = pd.DataFrame()
data['item_sets'] = count_ind_item.keys()
data['supp_count'] = count_ind_item.values()
data = data.sort_values('item_sets')
return data

```

```

def join(list_of_items):

```

```

    itemsets = []
    i = 1
    for entry in list_of_items:
        proceeding_items = list_of_items[i:]
        for item in proceeding_items:

```



```

    if(type(item) is str):
        if entry != item:
            tuples = (entry, item)
            itemsets.append(tuples)
        else:
            if entry[0:-1] == item[0:-1]:
                tuples = entry+item[1:]
                itemsets.append(tuples)
            i = i+1
    if(len(itemsets) == 0):
        return None
    return itemsets

In [6]:
def apriori(trans_data,supp=3, con=0.5):
    freq = pd.DataFrame()

    df = count_item(trans_data)

    while(len(df) != 0):

        df = prune(df, supp)

        if len(df) > 1 or (len(df) == 1 and int(df.supp_count) >= supp):
            freq = df

        itemsets = join(df.item_sets)

    if(itemsets is None):
        return freq

```

```
df = count_itemset(trans_data, itemsets)
```

```
return df
```

In [7]:

```
freq_item_sets = apriori(trans_df, 4)
```

```
freq_item_sets
```

Out[7]:

	item_sets	supp_count
15	(BREAD, SUGER)	4
16	(BREAD, TEA)	4
17	(COFFEE, CORNFLAKES)	4
19	(COFFEE, SUGER)	4
26	(MAGGI, TEA)	4

In [8]:

```
def calculate_conf(value1, value2):
```

```
    return round(int(value1)/int(value2) * 100, 2)
```

In [9]:

```
def strong_rules(freq_item_sets, threshold):
```

```
    confidences = {}
```

```
    for row in freq_item_sets.item_sets:
```

```
        for i in range(len(row)):
```

```
            for j in range(len(row)):
```

```
                if i != j:
```

```
                    tuples = (row[i], row[j])
```

```
                    conf = calculate_conf(freq_item_sets[freq_item_sets.item_sets == row].supp_count,
count_item(trans_df)[count_item(trans_df).item_sets == row[i]].supp_count)
```

```
confidences[tuples] = conf
```

```
conf_df = pd.DataFrame()
```

```
conf_df['item_set'] = confidences.keys()
```

```
conf_df['confidence'] = confidences.values()
```

```
return conf_df[conf_df.confidence >= threshold]
```

In [10]:

```
strong_rules(freq_item_sets, 50.0)
```

Out[10]:

	item_set	confidence
1	(SUGER, BREAD)	66.67
3	(TEA, BREAD)	57.14
4	(COFFEE, CORNFLAKES)	50.00
5	(CORNFLAKES, COFFEE)	66.67
6	(COFFEE, SUGER)	50.00
7	(SUGER, COFFEE)	66.67
8	(MAGGI, TEA)	80.00
9	(TEA, MAGGI)	57.14

Rules with confidence level >= 50.0%

In [11]:

```
linkcode
```

```
from functools import reduce # Valid in Python 2.6+, required in Python 3
```

```
import operator
```

```
def interesting_rules(freq_item_sets):
```

```
    lifts = {}
```

```
    prob_of_items = []
```

```
    for row in freq_item_sets.item_sets:
```

```
        num_of_items = len(row)
```

```
        prob_all = freq_item_sets[freq_item_sets.item_sets == row].supp_count / 18
```

```
        for i in range(num_of_items):
```

```
            prob_of_items.append(count_item(trans_df)[count_item(trans_df).item_sets ==
row[i]].supp_count / 18)
```

```
        lifts[row] = round(float(prob_all / reduce(operator.mul, (np.array(prob_of_items), 1)), 2)
```

```
    prob_of_items = []
```

```
    lifts_df = pd.DataFrame()
```

```
    lifts_df['Rules'] = lifts.keys()
```

```
    lifts_df['lift'] = lifts.values()
```

```
    return lifts_df
```

```
In [12]:
```

```
int_rules = interesting_rules(freq_item_sets)
```

```
int_rules
```

Apriori Algorithm Program in Python from Scratch – Jazib (<https://japp.io/machine-learning/apriori-algorithm-program-in-python-from-scratch/>)

```
data = [
    ['T100',['I1','I2','I5']],
    ['T200',['I2','I4']],
    ['T300',['I2','I3']],
    ['T400',['I1','I2','I4']],
    ['T500',['I1','I3']],
    ['T600',['I2','I3']],
    ['T700',['I1','I3']],
    ['T800',['I1','I2','I3','I5']],
    ['T900',['I1','I2','I3']]
]
```

```
init = []
for i in data:
    for q in i[1]:
        if(q not in init):
            init.append(q)
init = sorted(init)
print(init)
```

```
sp = 0.4
s = int(sp*len(init))
s
```

```
from collections import Counter
```

```
c = Counter()
```

```

for i in init:
    for d in data:
        if(i in d[1]):
            c[i]+=1
print("C1:")
for i in c:
    print(str([i])+"": "+str(c[i]))
print()
l = Counter()
for i in c:
    if(c[i] >= s):
        l[frozenset([i])]+=c[i]
print("L1:")
for i in l:
    print(str(list(i))+"": "+str(l[i]))
print()
pl = l
pos = 1
for count in range (2,1000):
    nc = set()
    temp = list(l)
    for i in range(0,len(temp)):
        for j in range(i+1,len(temp)):
            t = temp[i].union(temp[j])
            if(len(t) == count):
                nc.add(temp[i].union(temp[j]))
nc = list(nc)
c = Counter()
for i in nc:

```

```

c[i] = 0
for q in data:
    temp = set(q[1])
    if(i.issubset(temp)):
        c[i]+=1
print("C"+str(count)+":")
for i in c:
    print(str(list(i))+": "+str(c[i]))
print()
l = Counter()
for i in c:
    if(c[i] >= s):
        l[i]+=c[i]
print("L"+str(count)+":")
for i in l:
    print(str(list(i))+": "+str(l[i]))
print()
if(len(l) == 0):
    break
pl = l
pos = count
print("Result: ")
print("L"+str(pos)+":")
for i in pl:
    print(str(list(i))+": "+str(pl[i]))
print()

from itertools import combinations
for l in pl:

```

```

c = [frozenset(q) for q in combinations(l, len(l)-1)]
mmax = 0
for a in c:
    b = l-a
    ab = l
    sab = 0
    sa = 0
    sb = 0
    for q in data:
        temp = set(q[1])
        if(a.issubset(temp)):
            sa+=1
        if(b.issubset(temp)):
            sb+=1
        if(ab.issubset(temp)):
            sab+=1
    temp = sab/sa*100
    if(temp > mmax):
        mmax = temp
    temp = sab/sb*100
    if(temp > mmax):
        mmax = temp
    print(str(list(a))+" -> "+str(list(b))+" = "+str(sab/sa*100)+"%")
    print(str(list(b))+" -> "+str(list(a))+" = "+str(sab/sb*100)+"%")
curr = 1
print("choosing:", end=' ')
for a in c:
    b = l-a
    ab = l

```



```
sab = 0
sa = 0
sb = 0
for q in data:
    temp = set(q[1])
    if(a.issubset(temp)):
        sa+=1
    if(b.issubset(temp)):
        sb+=1
    if(ab.issubset(temp)):
        sab+=1
temp = sab/sa*100
if(temp == mmax):
    print(curr, end = ' ')
curr += 1
temp = sab/sb*100
if(temp == mmax):
    print(curr, end = ' ')
curr += 1
print()
print()
```